

# Spring AOP — Quick Revision Summary

## 1. Business Method

A method that does the core job of the app (like saving a student). It should only contain business logic, nothing else.

## 2. Real Apps Need More Than Business Logic

Apps also need logging, security, auditing, transactions, caching, etc. These are not part of core business logic.

## 3. Business Logic vs Supporting Logic

Business logic = domain rules (create student, place order). Supporting logic = logging, security, transactions — same across many features.

## 4. Concern

Any responsibility/area of interest in the app (e.g., logging, security, student management).

### Core Business Concerns

Concerns tied to one specific feature/module (StudentService, OrderService). Organized **vertically**.

### Cross-Cutting Concerns

Concerns needed across many unrelated modules (logging, security, auditing). Organized **horizontally**.

## 5. Common Cross-Cutting Examples

- **Logging** - record method start/end/success/failure.
- **Security** - check permission before an action.
- **Performance** - measure execution time.
- **Auditing** - record who/what/when/success-failure.
- **Transaction Management** - begin/commit/rollback DB transaction.

## 6. Scattering & Tangling (The Two Big Problems)

- **Scattering** - same concern's code repeated in many classes (logging in every service).
- **Tangling** - one method mixes multiple concerns together (business + logging + security + auditing all in one method), making it messy.

## 7. First Attempt: Utility Methods

Put common code (like logging) in a utility class. Problem: you still have to manually call it in every method — invocation is still scattered.

## 8. Second Attempt: Inheritance

Put common methods in a base class, services extend it. Problem: still needs manual calls, Java allows only single inheritance, forces artificial class hierarchy.

## 9. Third Attempt: Manual Wrappers (Decorator)

Create a wrapper class implementing same interface as real service; wrapper adds logging/security before & after calling the real object. This is close to how Spring AOP works internally, but doing it manually is very close to AOP's idea.

## 10. Limitations of Manual Wrappers

Need a separate wrapper for every service type; multiple concerns = multiple nested wrapper layers; hard to build/maintain at scale.

## 11. What a Better Solution Needs

1. Keep common behaviour in one place.
2. Decide *where* it applies (which methods).
3. Decide *when* it runs (before/after/on success/on failure).
4. Don't touch every business method manually.

## 12. Introduction to AOP

AOP = keep business logic separate + keep cross-cutting logic separate + define where/when it applies → combined automatically at runtime.

## 13. OOP vs AOP

- **OOP** organizes app **vertically** by business capability (StudentService, OrderService).
- **AOP** organizes app **horizontally** by shared behaviour (logging, security across all services). They work together, not against each other.

## 14. Three Fundamental Questions in AOP

- **What should happen?** → **Advice** (the actual code, e.g. logging).
- **Where should it happen?** → **Pointcut** (rule selecting methods).
- **When should it happen?** → **Advice type** (before/after/around/on success/on exception).

## 15. Aspect

A module that bundles together: the advice (what), the pointcut (where), and the timing (when) — i.e., a full cross-cutting concern as one unit.

## 16. How Spring AOP Applies an Aspect

- **Target** = real object with business logic.
- **Proxy** = wrapper object Spring creates automatically around the target.
- Caller talks to the proxy → proxy runs cross-cutting logic before/after → then calls real target method.

## 17. Manual Wrapper vs Spring AOP

Spring AOP automates what manual wrappers do by hand: Spring creates proxies, manages them via IoC container, and pointcuts decide where advice applies — no manual wrapper-building needed.

## 18. Key Terms Cheat Sheet

| Term                  | Meaning                                             |
|-----------------------|-----------------------------------------------------|
| Concern               | A responsibility/area of interest                   |
| Cross-cutting concern | Concern needed across many unrelated modules        |
| Aspect                | Module representing a cross-cutting concern         |
| Advice                | The actual action (logging, timing, etc.)           |
| Pointcut              | Rule deciding where advice runs                     |
| Target                | Real Spring bean with business logic                |
| Proxy                 | Wrapper Spring creates to intercept calls to target |

## 19. Final Mental Model

**Without AOP:** business method has logging + security + performance + auditing + real logic all mixed together (messy). **With AOP:** business class has only real logic; separate aspect holds logging/security/auditing rules; Spring proxy combines both at runtime.

Runtime flow: Caller → Proxy → Advice runs → Target method runs → remaining advice runs → result returns to caller.

## 20. Key Takeaways (Golden Points)

- Business methods should show only business intent.
- Logging, security, auditing, transactions = cross-cutting concerns.
- Mixing them in directly = scattering + tangling.
- Utility methods & inheritance don't fully solve it (still manual calls).
- Manual wrappers work but don't scale.
- AOP = separate module for cross-cutting logic + auto-apply via proxy.
- Advice = what, Pointcut = where, Advice type = when, Aspect = all three combined.
- OOP = vertical organization, AOP = horizontal organization.